

HEYAMA · TEST TECHNIQUE DEV

Plateforme de gestion d'Objects

Documentation technique complète : API · Web · Mobile · Bonus

Auteur Albert Le Grand

Dépôt github.com/legrandbakobogisdev/heyama-test

Version v1.0.0

Stack NestJS · MongoDB · MinIO · Socket.io · Next.js · shadcn/ui · Expo/React Native

Date 21 juillet 2026

1. Contexte et objectifs

Ce document décrit l'implémentation complète du test technique Heyama : un mini-système full-stack démontrant la communication entre une API centrale et deux clients (mobile et web), avec upload de fichiers vers un stockage objet S3-compatible et synchronisation temps réel entre tous les clients connectés.

Le sujet précisait explicitement que **le design UI n'est pas évalué** ("just make sure it's functional and coherent") et que **les solutions partielles sont acceptées**. La priorité a donc été donnée, dans l'ordre :

1. Une API robuste et complète (socle non négociable)
2. Une synchronisation temps réel fonctionnelle
3. Une couverture mobile **et** web, même minimaliste

Entité métier : **Object**

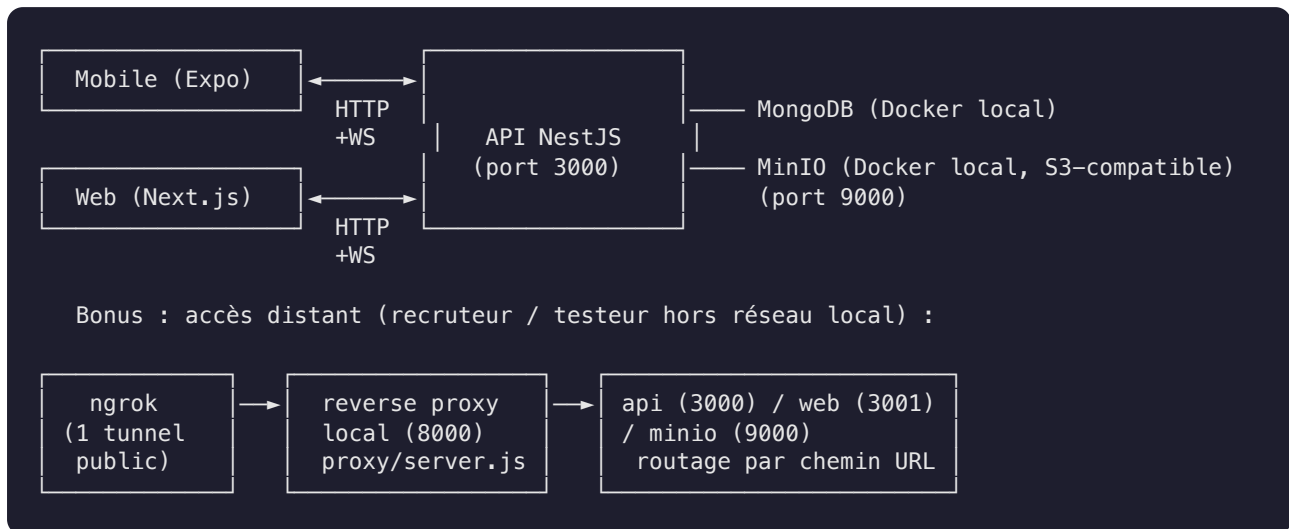
Champ	Type	Règle
<code>_id</code>	ObjectId (Mongo)	généré automatiquement
<code>title</code>	string	requis, non vide
<code>description</code>	string	requis
<code>imageUrl</code>	string	URL publique retournée après upload
<code>createdAt</code>	Date	généré automatiquement à la création

Fonctionnalités couvertes

- Créer un Object (titre + description + image) depuis mobile OU web
- Lister les Objects (mobile + web)
- Voir le détail d'un Object (mobile + web)
- Supprimer un Object, avec confirmation (mobile + web)
- Toute création/suppression se propage en temps réel sur tous les écrans connectés, via Socket.io

Le cahier des charges complet, avec son historique de décisions (changelog), est versionné dans le dépôt à [docs/CDC.md](#) . Ce document-ci en est la traduction "as-built" : ce qui a réellement été livré, fichier par fichier.

2. Architecture générale



Flux de données typique (création d'un Object)

1. Le client (web ou mobile) envoie un `POST /objects` en `multipart/form-data` (titre, description, image)
2. L'API valide le DTO (`class-validator`), rejette si titre/description vides ou image absente/invalid
3. L'image est uploadée vers MinIO via le SDK `@aws-sdk/client-s3` (compatible S3)
4. Le document est sauvegardé en MongoDB via Mongoose
5. Le `ObjectsGateway` (Socket.io) émet l'événement `object:created` à tous les clients connectés
6. Chaque client (web/mobile) reçoit l'événement et insère le nouvel Object en tête de liste, sans refetch complet

3. Choix techniques et justifications

Composant	Choix	Justification
API	NestJS	Imposé par le sujet
DB	MongoDB (Mongoose), Docker local	Imposé ; local = zéro dépendance réseau externe
Stockage image	MinIO (S3-compatible, Docker local)	Voir §3.1 ci-dessous
Realtime	Socket.io (@nestjs/websockets)	Imposé explicitement
Web	Next.js 16 (App Router) + shadcn/ui	Imposé
Mobile	Expo (React Native) + Expo Router	Imposé
Branching	GitFlow (git-flow CLI)	Demande explicite du candidat

3.1 · Pourquoi MinIO plutôt que Cloudflare R2

Le choix initial (documenté v1.0 du CDC) était Cloudflare R2, sur l'hypothèse qu'un stockage local type MinIO générerait des URLs `http://localhost:9000/...` injoignables depuis un smartphone physique. Ce raisonnement s'est avéré incomplet : **ce problème n'est pas spécifique à MinIO** : l'API NestJS elle-même tourne en local et a exactement le même souci d'accessibilité depuis un device physique. Sa solution (utiliser l'IP locale du PC hôte, `http://192.168.x.x:PORT`, tant que le mobile est sur le même Wi-Fi) s'applique identiquement à MinIO.

Le pivot vers MinIO a donc été acté en cours de développement (voir [docs/CDC.md](#) §3.2 pour le changelog complet), pour plusieurs raisons :

- Zéro compte externe à créer, zéro clé API cloud à générer
- Respecte davantage l'esprit "no deployment needed" du sujet : tout tourne en local
- Un seul `docker-compose.yml` (Mongo + MinIO) pour tout le setup

3.2 · GitFlow

Le dépôt suit un workflow GitFlow strict via l'extension `git-flow` : branche `main` réservée aux releases taguées, `develop` comme branche d'intégration, et une branche `feature/*` par unité de travail (scaffold API, scaffold web, scaffold mobile, downgrade SDK, reverse proxy ngrok...), fusionnée dans `develop` via `git flow feature finish` une fois validée. Le rendu final a été livré via un `git flow release` (`release/v1.0.0`) fusionné dans `main` et tagué `v1.0.0` .

4. Structure du dépôt

```
.
├── api/           # NestJS – API centrale
├── web/           # Next.js – client web
├── mobile/        # Expo/React Native – client mobile
├── proxy/         # Reverse proxy local (bonus, exposition ngrok)
├── docs/
│   └── CDC.md     # Cahier des charges + changelog des décisions
├── docker-compose.yml # MongoDB + MinIO en local
├── PRODUCT.md     # Contexte produit (register, personas, principes) pour le pass
└── README.md     # Instructions de setup
```

5. Backend · API NestJS (`api/`)

5.1 · Point d'entrée et configuration

`src/main.ts` Bootstrap de l'application NestJS. Active le CORS large (`origin: '*'` , requis pour que web/mobile sur des origines différentes puissent appeler l'API) et le `ValidationPipe` global (`whitelist: true`) qui applique automatiquement les décorateurs `class-validator` des DTO.

`src/app.module.ts` Module racine. Importe `ConfigModule` (variables d'environnement, `isGlobal: true`), `MongooseModule.forRoot(MONGO_URI)` (connexion Mongo), et `ObjectsModule` (le cœur métier).

5.2 · Module `objects/` (CRUD principal)

`src/objects/schemas/object.schema.ts` Schéma Mongoose de l'entité `ObjectEntity` : `title` , `description` , `imageUrl` (tous requis), `createdAt` (généré via `Date.now`).

`src/objects/dto/create-object.dto.ts` DTO de validation pour `POST /objects` : `title` et `description` décorés `@IsNotEmpty()` . L'image n'est pas dans le DTO (elle transite via Multer, gérée séparément dans le contrôleur).

`src/objects/objects.controller.ts` Expose les 4 routes REST :

Méthode	Route	Comportement
POST	<code>/objects</code>	<code>FileInterceptor</code> avec filtre MIME (<code>image/jpeg</code> , <code>image/png</code> , <code>image/webp</code>) et limite 5 Mo ; rejette en <code>400</code> si type invalide ou fichier absent
GET	<code>/objects</code>	Liste triée par <code>createdAt</code> décroissant
GET	<code>/objects/:id</code>	<code>200</code> + objet, ou <code>404</code>
DELETE	<code>/objects/:id</code>	<code>204</code> , ou <code>404</code>

`src/objects/objects.service.ts` Logique métier : orchestre l'upload vers `StorageService`, la persistance Mongo, et l'émission des events Socket.io via `ObjectsGateway` après chaque mutation réussie. La suppression retire **d'abord** le fichier sur MinIO puis le document Mongo, pour éviter des documents orphelins pointant vers un fichier déjà supprimé en cas d'erreur partielle.

`src/objects/objects.gateway.ts` Gateway Socket.io (`@WebSocketGateway({ cors: { origin: '*' } })`). Un seul namespace/room global (pas de scoping par utilisateur, hors périmètre du test). Deux méthodes : `emitCreated` et `emitDeleted`, appelées par le service après chaque mutation.

`src/objects/objects.module.ts` Assemble le tout : enregistre le schéma Mongoose via `MongooseModule.forFeature`, importe `StorageModule`, déclare le contrôleur et les providers (service + gateway).

5.3 · Module `storage/` (abstraction S3/MinIO)

`src/storage/storage.service.ts` Encapsule tous les échanges avec le stockage objet via `@aws-sdk/client-s3` :

```
new S3Client({
  region: 'us-east-1',
  endpoint: process.env.S3_ENDPOINT,
  forcePathStyle: true, // requis par MinIO (style de requête path/bucket)
  credentials: {
    accessKeyId: process.env.S3_ACCESS_KEY,
    secretAccessKey: process.env.S3_SECRET_KEY,
  },
})
```

Trois méthodes : `upload()` (génère une clé aléatoire via `randomUUID()`, préserve l'extension d'origine, renvoie l'URL publique + la clé), `delete()`, et `keyFromUrl()` (retrouve la clé S3 à partir de l'URL stockée en base, nécessaire pour la suppression).

Cette abstraction est ce qui a permis le pivot Cloudflare R2 → MinIO en changeant uniquement les variables d'environnement (`S3_ENDPOINT`, `S3_PUBLIC_URL`) et l'ajout de `forcePathStyle: true` : aucun autre fichier du projet n'a eu besoin d'être modifié.

5.4 · Variables d'environnement (`api/.env`)

```
S3_ENDPOINT=http://localhost:9000      # utilisé côté serveur uniquement
S3_ACCESS_KEY=minioadmin
S3_SECRET_KEY=minioadmin
S3_BUCKET=heyama-objects
S3_PUBLIC_URL=http://localhost:9000    # ou IP locale / URL ngrok pour accès distant
MONGO_URI=mongodb://localhost:27017/heyama
PORT=3000
```

6. Frontend Web · Next.js (`web/`)

Stack : Next.js 16 (App Router, Turbopack), TypeScript, Tailwind CSS v4, shadcn/ui (variante "base-nova" sur Base UI : attention, ce n'est pas Radix : le pattern de composition utilise `render` au lieu de `asChild`).

6.1 · Pages (App Router)

`src/app/layout.tsx` Layout racine partagé par les 3 pages : header commun ("Heyama Objects", lien vers l'accueil), polices Geist, métadonnées globales.

`src/app/page.tsx` · Route `/` (liste) Client Component. Au montage : `fetchObjects()` pour l'état initial, puis connexion `socket.io-client` écoutant `object:created` (prepend en state local, pas de refetch) et `object:deleted` (filter). Grille responsive de `Card` (image + titre), état de chargement via `Skeleton`, état vide avec CTA "Créer le premier".

`src/app/objects/new/page.tsx` · Route `/objects/new` Formulaire contrôlé (titre, description, input file avec `accept` restreint aux types acceptés par l'API). Soumission : `POST` multipart direct vers l'API (pas de Server Action), puis `router.push('/')`.

`src/app/objects/[id]/page.tsx` · Route dynamique détail Fetch par id au montage (`useParams`), affichage image/titre/description/date, bouton "Supprimer" avec `window.confirm` puis `router.back()` / redirection liste.

6.2 · Bibliothèques partagées

`src/lib/api.ts` Client HTTP minimaliste (fetch natif) : `fetchObjects`, `fetchObject`, `createObject` (construit le `FormData`), `deleteObject`. Toutes les fonctions utilisent `NEXT_PUBLIC_API_URL` (par défaut `http://localhost:3000`).

`src/lib/socket.ts` Singleton `socket.io-client` partagé entre les pages (`getSocket()`), évite les reconnexion multiples lors des changements de route.

6.3 · Design system et polish

Le sujet précise que le design n'est pas évalué ; un pass de polish a néanmoins été appliqué (à la demande du candidat) via le skill *impeccable*, avec un `PRODUCT.md` définissant un register "product" sobre et technique :

- Accent couleur unique et contrôlé (bleu, `oklch(0.5 0.17 259)`) introduit sur un thème shadcn initialement 100% neutre (chroma 0)
- Remplacement des couleurs codées en dur (`zinc-50` , `red-600` ...) par les tokens du design system (`bg-background` , `text-muted-foreground` , `text-destructive`)
- Navigation cohérente entre les 3 pages (header partagé, liens "← Retour")
- États vides et de suppression avec confirmation explicite

Bug corrigé en cours de route : le composant `Button` de shadcn (variante Base UI) plantait au runtime quand on lui passait un `<Link>` via `render` sans `nativeButton={false}` : Base UI attend un `<button>` natif par défaut, contrairement à Radix. Corrigé sur les deux boutons de navigation concernés.

6.4 · Configuration réseau

```
# web/.env.local
NEXT_PUBLIC_API_URL=http://localhost:3000    # ou IP locale / URL ngrok
NEXT_PUBLIC_BASE_PATH=                      # vide en local, "/app" derrière le proxy
```

Le serveur de dev tourne sur le port **3001** (et non 3000, déjà pris par l'API), voir `package.json` :

```
"dev": "next dev -p 3001" .
```

7. Mobile • Expo / React Native (`mobile/`)

Stack : Expo SDK 54 (managed workflow), Expo Router (navigation par fichiers, Stack simple : pas de tabs), TypeScript, `expo-image-picker`, `socket.io-client`.

Pivot SDK 57 → SDK 54 : le scaffold initial (`create-expo-app@latest`) a généré un projet SDK 57, incompatible avec la version d'Expo Go disponible sur le Play Store à ce moment (SDK 54). Le projet a été rétrogradé via `npx expo install expo@54 + npx expo install --fix` + réinstallation propre (`node_modules` / `package-lock.json` supprimés) pour aligner toutes les dépendances (React 19.1.0, React Native 0.81.5, Expo Router ~6.0.24, etc). Deux ajustements de code ont suivi ce downgrade : `DarkTheme` / `DefaultTheme` / `ThemeProvider` ne sont plus ré-exportés par `expo-router` en SDK 54 (import direct depuis `@react-navigation/native`), et `useColorScheme()` ne renvoie plus la valeur `'unspecified'` en RN 0.81.

7.1 • Navigation

`src/app/_layout.tsx` Remplace le layout à onglets ("tabs") généré par défaut par un `Stack` simple expo-router à 3 écrans (`index`, `objects/new`, `objects/[id]`), conforme au besoin CDC ("navigation simple"). Conserve le splash screen animé du template (`AnimatedSplashOverlay`).

7.2 • Écrans

`src/app/index.tsx` • `ObjectList` `FlatList` des Objects (image + titre), bouton flottant "+" (`Link asChild` vers `/objects/new` : expo-router supporte `asChild`, à la différence de Base UI côté web), connexion socket.io identique au web (prepend/filter en state local).

`src/app/objects/new.tsx` • `ObjectCreate` Formulaire identique au web. Sélection d'image via `expo-image-picker` :

```
const result = await ImagePicker.launchImageLibraryAsync({
  mediaTypes: ['images'], // API courante ; MediaTypeOptions est dépréciée
  quality: 0.8,
});
```

Construction d'un `FormData` avec `{ uri, name, type }` (format spécial React Native) pour l'upload multipart.

`src/app/objects/[id].tsx` · `ObjectDetail` Détail de l'Object, suppression via `Alert.alert` (confirmation native) avant l'appel `deleteObject` .

7.3 · Bibliothèques partagées

`src/lib/api.ts` et `src/lib/socket.ts` : mêmes responsabilités que leurs équivalents web, adaptés au `FormData` React Native. URL API exposée via `EXPO_PUBLIC_API_URL` (préfixe requis par Expo pour les variables inlineées côté client).

IP locale, pas `localhost` : le téléphone physique ne peut pas résoudre `localhost` vers le PC hôte. `EXPO_PUBLIC_API_URL` doit pointer vers l'IP locale du Mac sur le Wi-Fi (ex. `http://192.168.1.100:3000`), ou vers l'URL ngrok pour un accès hors réseau local.

7.4 · Build APK release (bonus)

Génération du dossier natif Android et build via Gradle :

```
cd mobile
npx expo prebuild --platform android
cd android
./gradlew assembleRelease
# APK : android/app/build/outputs/apk/release/app-release.apk
```

Incompatibilité Gradle/AGP rencontrée : le wrapper Gradle généré par `expo prebuild` pointait vers Gradle 8.6, alors que le plugin Gradle de React Native (SDK 54) résout AGP 8.11.0 / Kotlin 2.1.20, qui exigent Gradle 8.13+. Erreur observée : `Unresolved reference 'extensions'/'extra'` dans `expo-modules-autolinking` (référence à une API interne de Gradle absente en 8.6). Corrigé en bumpant `gradle-wrapper.properties` vers `gradle-8.13-bin.zip` .

L'APK release est signé avec le keystore *debug* par défaut du template Expo : suffisant pour un envoi à un testeur, insuffisant pour une publication Play Store (nécessiterait un keystore de production dédié). L'URL de l'API (`EXPO_PUBLIC_API_URL`) est figée dans le bundle JS au moment du build : pour un testeur hors réseau local, elle doit pointer vers l'URL ngrok publique (voir §8.2) **avant** de lancer `gradlew assembleRelease` .

8. Fonctionnalités bonus

8.1 · Récapitulatif des bonus livrés

Bonus	Statut
Workflow GitFlow complet (feature branches, release, tag)	Fait
Design polish (accent color, tokens, états vides/erreur, confirmation suppression)	Fait
Accès distant (hors réseau local) via ngrok + reverse proxy	Fait
Build APK release autonome (gradlew)	Fait
PUT /objects/:id (édition)	Non fait (hors scope strict du CDC)
UI de suppression bonus (bouton sur la liste)	Non fait : suppression uniquement en détail
Tests automatisés (e2e)	Squelette Jest généré par défaut, non étendu

8.2 · Accès distant via ngrok (reverse proxy)

Le plan gratuit ngrok n'accorde **qu'un seul hostname public par session/compte** : tenter d'ouvrir 3 tunnels simultanés (api, web, minio) aboutit à ce que les 3 partagent le même hostname et se marchent dessus (dernier tunnel démarré = seul qui répond réellement). La solution retenue : un unique tunnel ngrok vers un petit reverse proxy local qui redistribue par chemin d'URL.

`proxy/server.js` Serveur HTTP Node (module `http-proxy`) sur le port 8000 :

- `/` (défaut) → API (localhost:3000) : inclut aussi le handshake Socket.io (`/socket.io/...`), qui n'a besoin d'aucun préfixe particulier
- `/app*` → Web (localhost:3001) : **sans** retrait du préfixe : Next.js gère lui-même son `basePath` en interne (une des deux itérations du proxy retirait le préfixe par erreur, cassant la racine `/app` exact : corrigé)
- `/uploads/*` → MinIO (localhost:9000), avec retrait du préfixe `/uploads`

Supporte aussi l'upgrade WebSocket (`proxy.ws()`) pour Socket.io.

Côté Next.js, le `basePath` est rendu configurable pour ne pas impacter le développement local normal :

```
// web/next.config.ts
basePath: process.env.NEXT_PUBLIC_BASE_PATH || "",
```

Procédure de mise en ligne :

```
# 1. Config ngrok (~/.Library/Application Support/ngrok/ngrok.yml) : juste l'authtoken
# 2. Démarrer tous les services locaux (Mongo/MinIO, API, Web, proxy)
node proxy/server.js
# 3. Un seul tunnel, vers le proxy
ngrok http 8000
# 4. Reporter l'URL ngrok dans web/.env.local, mobile/.env (NEXT_PUBLIC_API_URL / EXPO_
#    et api/.env (S3_PUBLIC_URL=<url>/uploads), puis redémarrer API + Web
```

Le domaine ngrok gratuit observé (***.ngrok-free.dev**) s'est révélé **stable d'une session à l'autre** pour un même compte (contrairement à l'ancien comportement 100% aléatoire de ngrok v2) : il est donc possible d'éteindre la machine et de tout relancer plus tard (Docker → API → Web → proxy → ngrok, dans cet ordre) sans que l'URL ne change, et donc sans avoir à reconstruire l'APK.

9. Installation et lancement

9.1 · Prérequis

- Docker (Mongo + MinIO)
- Node.js 18+ / npm
- Pour le mobile : Expo Go (SDK 54) sur un téléphone, sur le même Wi-Fi que le PC

9.2 · Démarrage

```
# 1. Mongo + MinIO
docker compose up -d
# le service minio-init crée automatiquement le bucket "heyama-objects"
# et le rend public en lecture

# 2. API
cd api
cp .env.example .env
npm install
npm run start:dev                # http://localhost:3000

# 3. Web
cd web
npm install
npm run dev                      # http://localhost:3001

# 4. Mobile
cd mobile
npm install
npx expo start                  # scanner le QR avec Expo Go
```

9.3 · Variables d'environnement à adapter

Fichier	Variable	Valeur locale
api/.env	S3_PUBLIC_URL	IP locale du PC (pas localhost) si test mobile sur le même Wi-Fi
mobile/.env	EXPO_PUBLIC_API_URL	IP locale du PC (obligatoire, localhost ne fonctionne pas sur device physique)
web/.env.local	NEXT_PUBLIC_API_URL	http://localhost:3000 suffit en local

10. Tests et validations effectués

Vérification	Méthode	Résultat
Build/lint/typecheck API	<code>npm run build</code> , ESLint	OK, 0 erreur
Build/lint/typecheck Web	<code>next build</code> , <code>tsc --noEmit</code> , ESLint	OK, 0 erreur
Lint/typecheck Mobile	<code>expo lint</code> , <code>tsc --noEmit</code>	OK (2 erreurs cosmétiques pré-existantes du template, non bloquantes)
CRUD complet API (curl)	POST avec vraie image, GET liste/détail, DELETE	Upload MinIO + suppression fichier+doc vérifiés bout en bout
Validation métier	POST sans image → 400, POST sans titre → 400, GET id inconnu → 404	Conforme au CDC
Parcours web complet	Test manuel navigateur : création → apparition liste → détail → suppression	Validé par le candidat
Parcours mobile complet	Test manuel sur device physique (Expo Go)	Validé par le candidat, y compris la synchronisation temps réel
Accès distant ngrok	curl sur l'URL publique (API, web, uploads)	200 sur les 3 routes

11. Limitations connues et pistes d'amélioration

- **Pas d'authentification** : hors scope explicite du CDC
- **Pas de pagination/recherche** : la liste charge tous les Objects en une fois
- **Pas d'édition** (`PUT /objects/:id`) : seuls create/read/delete sont implémentés
- **Suppression uniquement depuis l'écran détail** : pas de bouton delete direct sur la liste
- **Tests automatisés minimaux** : le squelette Jest par défaut de NestJS n'a pas été étendu avec des tests e2e sur `POST /objects`
- **Socket.io sans scoping utilisateur** : un seul namespace/room global, cohérent avec l'absence d'authentification
- **URLs ngrok liées à une session locale active** : si la machine du candidat est éteinte et qu'aucun service n'est relancé, l'accès distant (web/API/images) est indisponible tant que les services ne sont pas redémarrés dans l'ordre documenté en §8.2